Mahalanobis-Anomalieerkennung – Modelldokumentation

Nov 9, 2026 · @Klaus

Zweck und Überblick

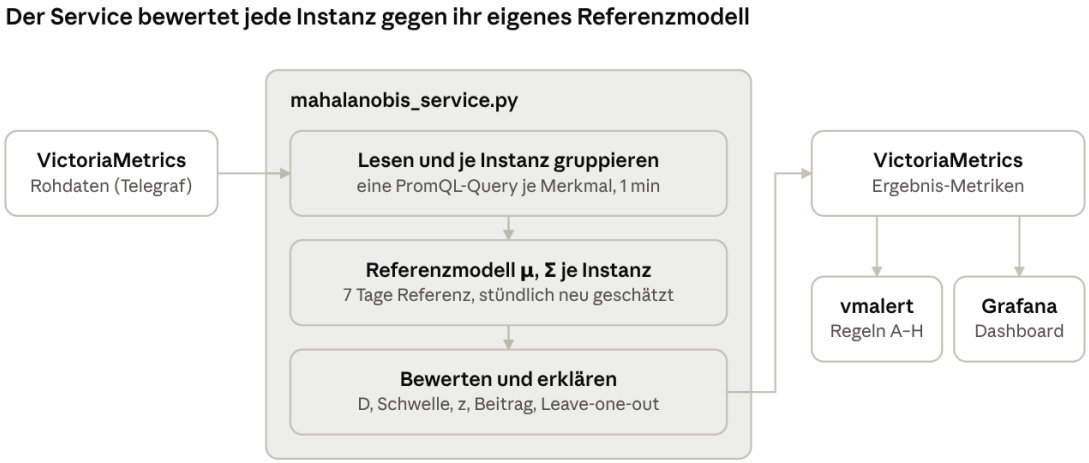
Das Modell erkennt für jede Server-Instanz Zeitpunkte, deren Kombination von Messwerten untypisch für den Normalbetrieb ist, und benennt die Merkmale, die dafür verantwortlich sind. Grundlage ist die Mahalanobis-Distanz: Sie misst den Abstand einer Messung zum Referenzbetrieb unter Berücksichtigung der Streuung und der Zusammenhänge zwischen den Merkmalen.

Dadurch erkennt das Modell zwei Arten von Auffälligkeiten:

- **Einzelmerkmal-Abweichungen:** Ein Messwert liegt für sich allein außerhalb des Normalbereichs, z. B. eine stark erhöhte I/O-Wartezeit.
- **Korrelationsbrüche:** Alle Einzelwerte liegen im Normalbereich, aber ihre Kombination kommt im Normalbetrieb nicht vor, z. B. hohe Systemlast ohne entsprechenden Netz- oder Disk-Verkehr. Diese Fälle erkennen einfache Schwellwerte je Metrik nicht.

Das Modell läuft als Service zwischen zwei Zugriffen auf VictoriaMetrics:

1. Rohdaten je Merkmal per PromQL lesen, getrennt nach Instanz.
2. Je Instanz ein Referenzmodell aus einem Zeitraum im Normalbetrieb schätzen.
3. Jeden neuen Messzeitpunkt bewerten und die Abweichung erklären.
4. Ergebnisse als Metriken mit dem Zeitstempel der Messung zurückschreiben, wo Alert-Regeln und ein Grafana-Dashboard sie auswerten.



Datenfluss · Lesen, Referenzmodell, Bewertung je Instanz

Die Rohdaten und die Ergebnisse liegen in derselben VictoriaMetrics; Alert-Regeln und Dashboard lesen nur die Ergebnis-Metriken.

Datei	Aufgabe
`mahalanobis_analyse.py`	Analysekern: Referenzmodell, Distanz, Schwelle, Erklärung
`vm_export.py`	Umwandlung der Ergebnisse ins Prometheus-Format und Versand an VictoriaMetrics
`mahalanobis_service.py`	Service: Lesen, Gruppieren je Instanz, Zeitfenster, Zustand, Fehlerbehandlung
`config.yml`	Merkmale (PromQL), Labels, Zeitfenster, Analyse-Parameter
`mahalanobis_alerts.yml`	Alert-Regeln für vmalert
`grafana/mahalanobis_dashboard.json`	Grafana-Dashboard für Ergebnisse und Rohdaten

Datengrundlage

Jedes Merkmal ist das Ergebnis einer PromQL-Query aus der Konfiguration, gelesen mit einer Auflösung von 1 Minute. Die Ergebnisse werden nach dem Label `instance` gruppiert, und jede Instanz wird mit einem eigenen Referenzmodell bewertet.

Aktuell konfigurierte Merkmale:

Merkmal	Query (gekürzt)	Einheit
`system_load5`	`avg by (instance) (system_load5)`	Load
`mem_used_percent`	`avg by (instance) (mem_used_percent)`	%
`diskio_io_wait`	`avg by (instance) (diskio_io_await)`	ms
`net_recv`	`avg by (instance) (rate(net_bytes_recv[5m]))`	Bytes/s
`net_sent`	`avg by (instance) (rate(net_bytes_sent[5m]))`	Bytes/s
`diskio_read`	`avg by (instance) (rate(diskio_read_bytes[5m]))`	Bytes/s
`diskio_write`	`avg by (instance) (rate(diskio_write_bytes[5m]))`	Bytes/s

Alle Queries filtern zusätzlich auf `instance=~"q556|q556d"`.

Anforderungen an die Queries:

- **Eine Zeitreihe je Instanz:** Metriken mit weiteren Labels (Interface, Disk) werden mit `avg by (instance)` zusammengefasst. Liefert eine Query mehrere Zeitreihen für eine Instanz, wird diese Instanz übersprungen und ein Fehler protokolliert.
- **Zähler nur mit `rate()`:** Der Rohwert eines Zählers bildet nur die verstrichene Zeit ab, nicht die Last. Ein Zähler-Reset nach einem Neustart würde als massive Anomalie erscheinen.

Umgang mit fehlenden Werten:

- Messzeitpunkte, bei denen ein Merkmal fehlt, werden nicht bewertet. Unendliche Werte zählen als fehlend.
- Referenzpunkte mit fehlenden Werten fließen nicht in das Referenzmodell ein.
- Fehlt einer Instanz ein Merkmal vollständig, wird sie übersprungen und eine Warnung protokolliert.

Referenzmodell

Der Normalbetrieb einer Instanz wird durch den Mittelwertvektor μ und die Kovarianzmatrix Σ ihrer Merkmale beschrieben. Beide werden aus einem Referenzzeitraum geschätzt, standardmäßig den letzten 7 Tagen bis 1 Stunde vor jetzt.

Referenzzeitraum:

- **Gleitend (Standard):** 7 Tage, endend 1 Stunde vor dem Lauf, stündlich neu geladen. Die Referenz passt sich langsamen Veränderungen an.
- **Fest:** ein bekannter Zeitraum im Normalbetrieb (`reference.start` und `reference.end`). Die Referenz bleibt unverändert, bis sie bewusst neu gesetzt wird.
- Instanzen mit weniger vollständigen Referenzpunkten als `min_points` werden nicht bewertet.

Klassische Schätzung (Standard)

Mittelwert und Stichproben-Kovarianz über alle n Referenzpunkte:

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \Sigma = \frac{1}{n-1}\sum_{i=1}^{n} (x_i-\mu)(x_i-\mu)^\top$$

Die Schätzung ist schnell und deterministisch. Sie setzt voraus, dass die Referenz keine größeren Störungen enthält, da diese die Kovarianz aufblähen und spätere ähnliche Störungen verdecken.

Robuste Schätzung (optional)

Mit `robust: true` werden μ und Σ mit der Minimum Covariance Determinant (MCD, scikit-learn `MinCovDet`) geschätzt. Das Verfahren wählt den Anteil `support_fraction` der Referenzpunkte, deren Kovarianz die kleinste Determinante hat, und schätzt das Modell nur daraus. Ausreißer in der Referenz beeinflussen das Ergebnis dadurch kaum.

- **Standardisierung:** Vor dem Fit wird jedes Merkmal auf Mittelwert 0 und Standardabweichung 1 gebracht und das Ergebnis danach zurücktransformiert. Das verbessert die numerische Stabilität bei Merkmalen in sehr unterschiedlichen Größenordnungen und ändert das Ergebnis nicht.
- **Reproduzierbarkeit:** Der Zufallsgenerator ist fest gesetzt (`random_state=0`), gleiche Daten ergeben dasselbe Modell.
- **Anteil `support_fraction`:** Der Standardwert von etwa 0,5 beschreibt bei Servern mit Tagesgang vor allem die ruhige Phase. Normale Tageslast gilt dann als anomal. Empfohlen sind `support_fraction: 0.9` zusammen mit `schwelle: empirisch`.
- Warnungen von scikit-learn über einen Anstieg der Determinante in Zwischenschritten werden unterdrückt. Sie betreffen verworfene Kandidaten, nicht das Endergebnis.

Konstante Merkmale

Ein Merkmal, das in der Referenz konstant ist (Standardabweichung ≤ $10^{-12} \cdot \max(1, |\text{Mittelwert}|)$), würde die Kovarianz singulär machen. Es wird aus der Distanz ausgeschlossen und stattdessen auf exakte Abweichung vom Referenzwert geprüft. Weicht es ab, steht es im Ergebnis unter `konstant_abweichung`.

Für die Invertierung von Σ wird die Pseudoinverse verwendet. Fast singuläre Kovarianzen, z. B. durch stark korrelierte Merkmale, führen dadurch nicht zum Abbruch.

Bereinigung der Referenz (optional)

Mit `reference.clean_quantile` werden deutliche Anomalien vor der Schätzung aus der Referenz entfernt. Im Test wurde eine dreistündige Störung in der Referenz vollständig entfernt (180 von 180 Punkten), bei nur 0,7 % entfernten Normalpunkten.

Das Verfahren arbeitet iterativ:

1. Modell (μ, Σ) aus den aktuell behaltenen Referenzpunkten schätzen, beim ersten Durchlauf aus allen.
2. Distanz D für alle Referenzpunkte berechnen.
3. Punkte mit D über der Wurzel des Quantils `clean_quantile` der χ^2-Verteilung entfernen. Ein Punkt kann in einem späteren Durchlauf wieder aufgenommen werden.
4. Wiederholen, bis sich nichts mehr ändert, höchstens `clean_iterations` Mal (Standard 3).

Die Bereinigung läuft einmal pro Neuladen der Referenz, nicht bei jedem Lauf. Sie verwendet dieselbe Schätzung wie die Analyse (klassisch oder robust). Die empirische Schwelle wird auf der bereinigten Referenz berechnet. Das Log nennt je Instanz die Zahl der entfernten Punkte; ab 10 % erscheint eine Warnung mit dem Hinweis, den Referenzzeitraum zu prüfen.

- **Hohe Schwelle wählen:** 0,999 oder 0,9999, nicht 0,99. Sonst fallen auch seltene, aber normale Spitzen heraus (z. B. ein nächtliches Backup), und das Modell wird zu eng.
- **Grenze:** Machen Störungen einen großen Teil der Referenz aus (grob ab 20 bis 30 %), verzerren sie schon das erste Modell so stark, dass sie nicht als Ausreißer auffallen. Dann hilft nur ein anderer Referenzzeitraum.
- Die Funktion steht auch einzeln zur Verfügung: `bereinige_referenz()` in `mahalanobis_analyse.py`.

Mahalanobis-Distanz und Schwellwert

Ein Messzeitpunkt gilt als anomal, wenn seine Mahalanobis-Distanz D die Schwelle überschreitet. Bei 7 Merkmalen und dem 99-%-Quantil liegt die Schwelle bei D = 4,30.

Für einen Messvektor x mit p Merkmalen gilt:

$$D^2 = (x-\mu)^\top \, \Sigma^{-1} \, (x-\mu) \qquad D = \sqrt{D^2}$$

Im Gegensatz zum euklidischen Abstand gewichtet D jede Abweichung mit der Streuung des Merkmals und berücksichtigt die Korrelationen. Eine Abweichung entlang eines typischen Zusammenhangs (mehr Last bei mehr Netzverkehr) zählt wenig, eine Abweichung quer dazu viel.

Schwellwert

Verfahren	Berechnung	Geeignet für
`chi2` (Standard)	Wurzel des Quantils q der χ^2-Verteilung mit p Freiheitsgraden	annähernd normalverteilte Referenzdaten
`empirisch`	Quantil q der Distanzen aller Referenzpunkte zum Modell	Daten mit Tagesgang, schiefe Verteilungen (Bytes/s), robuste Schätzung

Der Freiheitsgrad p ist die Zahl der Merkmale ohne konstante Merkmale. Beispiele für q = 0,99: p = 3 ergibt D = 3,37, p = 7 ergibt D = 4,30.

Die Schwelle legt die Fehlalarmrate je Messzeitpunkt fest: Bei q = 0,99 sind etwa 1 % der Punkte im Normalbetrieb über der Schwelle, bei 1-Minuten-Auflösung rund 14 Minuten pro Tag. Einzelne anomale Punkte sind deshalb kein Ereignis; erst ein anhaltend hoher Anteil löst einen Alarm aus (siehe Bewertung und Alarmierung).

Die `empirische` Schwelle hält die Rate von 1 % auf der Referenz auch dann ein, wenn die Daten nicht normalverteilt sind.

Voraussetzung dafür ist eine weitgehend saubere Referenz. Liegt mehr als der Anteil 1 − q der Referenzpunkte in einer Störung (bei q = 0,99 also mehr als 1 %), fällt das Quantil in die Störung, und die Schwelle steigt stark an. Im Test mit einer dreistündigen Störung in der Referenz (1,8 % der Punkte) wurden so nur noch 15 von 30 Störungsminuten erkannt. Die empirische Schwelle deshalb bei möglicherweise verschmutzter Referenz nur zusammen mit der Bereinigung verwenden.

Erklärbarkeit der Ergebnisse

Zu jedem Messzeitpunkt berechnet das Modell fünf Kennwerte, die zeigen, welches Merkmal wie stark zur Distanz beiträgt. Zusammen unterscheiden sie Einzelmerkmal-Abweichungen von Korrelationsbrüchen.

Kennwert	Bedeutung	Wertebereich
z-Wert	Abweichung des Merkmals für sich allein, in Standardabweichungen	beliebig, Vorzeichen = Richtung
Beitrag	Additiver Anteil des Merkmals an D^2; die Summe aller Beiträge ist genau D^2	kann negativ sein
Anteil	Beitrag geteilt durch D^2	Summe = 1
Leave-one-out	Rückgang von D^2, wenn das Merkmal weggelassen wird	≥ 0
Haupttreiber	Merkmal mit dem größten Leave-one-out-Wert, nur bei Anomalien	Name des Merkmals

Die Formeln, mit σ_j als Standardabweichung des Merkmals j im Referenzmodell und $D^2_{(-j)}$ als Distanz ohne Merkmal j:

```
z_j = \frac{x_j-\mu_j}{\sigma_j} \qquad c_j = (x_j-\mu_j)\,\big(\Sigma^{-1}(x-\mu)\big)_j \qquad \textstyle\sum_j c_j = D^2 \qquad \mathrm{loo}_j = D^2 - D^2_{(-j)}
```

Für $D^2_{(-j)}$ wird die Kovarianz ohne Merkmal j neu invertiert. Zeilen und Spalten aus Σ^{-1} zu streichen wäre falsch.

Interpretation

- **Hoher z-Wert und hoher Beitrag:** Das Merkmal ist für sich allein auffällig. Maßnahme: diese Messgröße prüfen.
- **Kleine z-Werte (unter 3), aber D über der Schwelle:** Korrelationsbruch. Die Leave-one-out-Werte zeigen, welche Merkmale beteiligt sind, oft zwei mit fast gleich hohen Werten. Maßnahme: das Zusammenwirken dieser Größen prüfen.
- **Negativer Beitrag:** Das Merkmal weicht in eine Richtung ab, die zu einem korrelierten Partner passt, und „erklärt" dessen Abweichung teilweise.
- Der Haupttreiber nennt bei einem Korrelationsbruch nur eines der beteiligten Merkmale. Deshalb immer die Leave-one-out-Werte gemeinsam betrachten.

Ausgabemetriken

Jeder bewertete Messzeitpunkt wird mit seinem eigenen Zeitstempel als Satz von Metriken nach VictoriaMetrics geschrieben. Alle Metriken tragen die festen Labels aus `output.labels` (z. B. `anlage`, `modell`) und das Label `instance` aus dem Query-Ergebnis.

Metrik	Zusätzliche Labels	Wert
`mahalanobis_d`	–	Distanz D
`mahalanobis_d2`	–	quadrierte Distanz D^2
`mahalanobis_schwelle_d`	–	Schwelle für D
`mahalanobis_anomalie`	–	1 = anomal, 0 = normal
`mahalanobis_z`	`merkmal`	z-Wert
`mahalanobis_beitrag`	`merkmal`	Beitrag zu D^2
`mahalanobis_anteil`	`merkmal`	Anteil an D^2 (fehlt bei $D^2 = 0$)
`mahalanobis_loo`	`merkmal`	Leave-one-out-Effekt
`mahalanobis_haupttreiber`	`merkmal`	1, nur bei Anomalie
`mahalanobis_konstant_abweichung`	`merkmal`	1, nur wenn ein in der Referenz konstantes Merkmal abweicht
`mahalanobis_messwert`	`merkmal`	Eingangswert, nur mit `mit_messwert:`

Metrik	Zusätzliche Labels	Wert
		`true`
`mahalanobis_input_<merkmal>`	–	Eingangswert, nur mit `write_input: true`

Der Service schreibt außerdem je Lauf Metriken über sich selbst, nur mit den festen Labels:

Metrik	Labels	Wert
`mahalanobis_service_instanzen`	`status` = ok, uebersprungen, fehler	Anzahl Instanzen im Lauf
`mahalanobis_service_laufdauer_sekunden`	–	Dauer des Laufs in Sekunden

Das Präfix `mahalanobis` ist über `output.prefix` einstellbar.

Betrieb des Service

Der Service läuft standardmäßig alle 5 Minuten und bewertet jeden Messzeitpunkt genau einmal, auch nach Unterbrechungen bis zu 1 Stunde.

Zeitfenster eines Laufs

- **Ende:** jetzt minus `delay` (1 Minute), abgerundet auf `step` (1 Minute). Der Abstand stellt sicher, dass spät eintreffende Rohdaten bereits vorliegen.
- **Start je Instanz:** der Zeitpunkt nach dem zuletzt erfolgreich verarbeiteten. Eine neue Instanz beginnt mit einem Intervall.
- **Nachholen:** Nach einer Unterbrechung werden höchstens `max_catchup` (1 Stunde) nachgerechnet. Instanzen, die länger keine Daten hatten, starten neu.
- **Zustand:** Mit `state_file` überlebt der zuletzt verarbeitete Zeitpunkt je Instanz einen Neustart.
- Die Läufe sind auf volle Intervalle ausgerichtet, bei 5 Minuten also :00, :05, :10 usw.

Referenz-Cache

Die Referenzdaten werden je Lauf nicht neu gelesen, sondern zwischengespeichert und alle `refresh` (1 Stunde) neu geladen. Schlägt das Neuladen fehl, rechnet der Service mit der bisherigen Referenz weiter. Lange Zeiträume werden in Teilabfragen von höchstens 10.000 Punkten je Zeitreihe gelesen.

Trennung von Referenz und Messung

Ein bewerteter Zeitpunkt liegt nie in seiner eigenen Referenz. Überlappen sich Referenz und Messfenster, etwa beim Nachholen nach einer Unterbrechung mit `max_catchup` größer oder gleich `end_offset`, wird die Referenz vor dem ersten bewerteten Zeitpunkt abgeschnitten. Im Normalbetrieb (`end_offset` 1 Stunde, Intervall 5 Minuten) liegen 55 Minuten zwischen Referenz-Ende und erstem bewerteten Punkt, die Referenz bleibt dann unverändert.

Ergänzung um vorhergehende Messpunkte (optional)

Mit `reference.include_recent: N` wird jeder Zeitpunkt t gegen eine eigene Referenz bewertet: die Referenzdaten plus die Messpunkte t−1 bis t−N. Standard ist 0 (aus).

- Damit auch der erste Zeitpunkt eines Laufs seine Vorgänger hat, liest der Service N Zeitpunkte vor dem Messfenster mit.
- Fehlende Vorgänger (Lücken, unvollständige Werte) werden ausgelassen; die Referenz ist dann um entsprechend weniger Punkte ergänzt.
- Vorgänger, die bereits in der Referenz liegen, werden nicht doppelt gezählt.
- Jeder Zeitpunkt wird einzeln gerechnet. Ein normaler 5-Minuten-Lauf mit robuster Schätzung und N = 3 dauerte im Test etwa 4,5 Sekunden für zwei Instanzen.

Zur Wirkung auf die Erkennung siehe Abschnitt „Keine aktuellen Messpunkte in der Referenz“.

Fehlerbehandlung

Situation	Verhalten
Merkmal fehlt für eine Instanz	Instanz übersprungen, Warnung im Log
Query liefert mehrere Zeitreihen je Instanz	Instanz übersprungen, Fehler im Log
Zu wenige Referenzpunkte (`min_points`)	Instanz übersprungen, Status `uebersprungen`
Analyse oder Schreiben für eine Instanz schlägt fehl	nur diese Instanz, Status `fehler`, Zeitfenster wird im nächsten Lauf wiederholt
VictoriaMetrics nicht erreichbar	Lauf abgebrochen, nächster Versuch im nächsten Intervall
Fehler in der Konfiguration	Start mit verständlicher Fehlermeldung abgebrochen

Ausführung

- Dauerbetrieb: `python mahalanobis_service.py -c config.yml`, beendet sich sauber bei SIGTERM.
- Einzelner Lauf: `--once`, z. B. für Cron. Exit-Code 0 = ok, 1 = Fehler bei Instanzen, 2 = Lauf fehlgeschlagen.
- Prüflauf ohne Schreiben: `--once --dry-run`.
- Voraussetzungen: Python 3.8 oder neuer, numpy, pandas 2, scipy, requests, PyYAML; scikit-learn für die robuste Schätzung. Alternativ der Docker-Container aus dem Paket.

Bewertung und Alarmierung

Ein Alarm entsteht erst, wenn mindestens die Hälfte der Messzeitpunkte der letzten 15 Minuten anomal ist und das 5 Minuten anhält. Einzelne anomale Punkte, die bei 1 % Fehlalarmrate normal sind, lösen nichts aus.

```
avg_over_time(mahalanobis_anomalie[15m]) >= 0.5      # mit for: 5m
```

Die Regeln in `mahalanobis_alerts.yml` (vmalert) ordnen über `ignoring(merkmal)` zu. Jede Instanz wird so getrennt bewertet, unabhängig davon, welche festen Labels konfiguriert sind. Die Regeln A und B schließen sich gegenseitig aus und ordnen jede anhaltende Anomalie einer Art zu.

Regel	Bedingung	Schwere	Handlung
A `MerkmalAusserhalbNormalbereich`	anhaltend anomal und ein Merkmal mit |z| ≥ 3; ein Alarm je Merkmal mit Richtung	warning	Messstelle bzw. Prozessgröße prüfen
B `MerkmalsZusammenhangGestoert`	anhaltend anomal, aber kein Merkmal mit |z| ≥ 3; nennt die 3 stärksten Treiber	warning	Zusammenwirken der Treiber prüfen
C `AnomalieKritisch`	anhaltend anomal und D ≥ 2 × Schwelle	critical	sofort reagieren
D `BetriebszustandAusserhalbReferenz`	ein in der Referenz konstantes Merkmal weicht ab	info	Bewertung für diesen Zustand nicht gültig; ggf. eigene Referenz
E `MahalanobisDrift`	6-h-Mittel von D über 1,5 × Mittel der Vorwoche, 1 h lang	info	Verschleiß oder veraltete Referenz prüfen
F `MahalanobisKeineDaten`	für eine Instanz seit 30 min keine Ergebnisse	warning	Rohdaten und Service-Log prüfen
G `MahalanobisServiceInaktiv`	seit 30 min kein Lauf des Service	warning	Prozess und Erreichbarkeit prüfen
H `MahalanobisServiceProbleme`	Instanzen mit Status fehler oder uebersprungen, 15 min lang	info	Service-Log prüfen

F und G nutzen die MetricsQL-Funktion `lag()` und laufen nur mit VictoriaMetrics. C feuert zusätzlich zu A bzw. B; eine Inhibit-Regel im Alertmanager kann die Warnungen bei kritischen Fällen unterdrücken.

Die Zeitfenster der Regeln (10 und 15 Minuten) müssen mindestens so lang sein wie Intervall plus Verzögerung des Service, da die Ergebnisse blockweise geschrieben werden.

Das Grafana-Dashboard zeigt dieselben Größen: Status und Anomalie-Anteil je Instanz, Distanz mit Schwelle, |z| je Merkmal, Beiträge, Leave-one-out und die Rohdaten mit Anomalien als rote Bereiche.

Validierung

In allen Testszenarien hat das Modell die eingebauten Störungen vollständig erkannt und den richtigen Treiber benannt. Getestet wurde mit synthetischen Daten gegen eine echte VictoriaMetrics-Instanz (v1.106.1); eine Auswertung mit echten Produktionsdaten steht noch aus.

Szenario	Erwartung	Ergebnis
Korrelationsbruch, 3 Merkmale (Temperatur 75, Druck 170 bei normalem Verhältnis 1:2)	anomal, obwohl jeder Einzelwert normal ist	D = 9,9 bei Schwelle 3,37; |z| ≈ 1,0; Temperatur und Druck je 50 % Anteil
Einzelmerkmal, Drehzahl 1700 statt 1500	anomal, Drehzahl als Ursache	D = 3,9; z = 3,9; Drehzahl 100 % Anteil
Service mit 4 Instanzen, 30 min Messfenster	Störungen auf 2 Instanzen erkannt, fehlerhafte Instanzen übersprungen	je 20 von 20 Störungsminuten anomal; gesunde Instanz 1 Fehlalarm; Instanz ohne Merkmal und mehrdeutige Query korrekt übersprungen
Server-Metriken im Telegraf-Format (7 Merkmale, 8 Tage, Tagesgang), q556d mit 30 min hoher Last und I/O-Wartezeit	Störung erkannt, gesunder Host ruhig	q556d 32 bis 33 Minuten anomal, Haupttreiber `diskio_io_wait` (20 min) und `diskio_write` (11 min); q556 1 Fehlalarm in 6 h
Alert-Regeln auf diesen Daten	nur die eingebauten Fälle	A nur host3/Drehzahl, B nur host2 mit Treibern Temperatur und Druck, C host2; keine weiteren Treffer
Netzwerk-Zähler ohne `rate()`	ungeeignet	z dauerhaft +1,7 bis +1,8 unabhängig von der Last; mit `rate()` folgt z der Last von −0,9 bis +1,5

Die 2 bis 3 Minuten über die 30-minütige Störung hinaus entstehen durch das 5-Minuten-Fenster von `rate()`, das die Störung nachlaufen lässt.

Klassische und robuste Schätzung im Vergleich

Auf den Telegraf-Testdaten, Messfenster 6 Stunden, Schwelle `chi2` außer in der letzten Zeile:

Schätzung	Referenzpunkte über Schwelle	davon tagsüber 10–18 Uhr	q556 gesund: Fehlalarme	q556d Störung: anomal
klassisch	1,8 %	4,7 %	1	33

Schätzung	Referenzpunkte über Schwelle	davon tagsüber 10–18 Uhr	q556 gesund: Fehlalarme	q556d Störung: anomal
robust, `support_fraction` Standard (≈ 0,5)	7,1 %	18,2 %	11 bis 15	38 bis 43
robust, `support_fraction: 0.9`	3,3 %	8,7 %	4 bis 9	34 bis 37
robust, 0,9 und `schwelle: empirisch`	1 % (per Definition)	–	0	32

Die Spannen bei den robusten Varianten entstehen durch unterschiedliche scikit-learn-Versionen. Mit klassischer Schätzung liefern Python 3.8 und 3.13 identische Ergebnisse.

Bereinigung bei gestörter Referenz

In die Referenz von q556d wurde vor 3 Tagen eine dreistündige Störung derselben Art eingebaut (180 Punkte, 1,8 % der Referenz). Ohne Bereinigung maskiert sie die aktuelle Störung; mit Bereinigung ist das Modell praktisch so gut wie mit sauberer Referenz.

Variante	entfernt (davon alte Störung)	aktuelle Störung erkannt	D Median	Fehlalarme in 5,5 h
saubere Referenz (Vergleich)	–	30/30	38,0	3
gestörte Referenz, ohne Bereinigung	0	30/30	8,3	3
robust 0,9 + empirisch, ohne Bereinigung	–	15/30	40,9	0
`clean_quantile: 0.999`	248 (180)	30/30	38,9	3
`clean_quantile: 0.9999`	199 (180)	30/30	38,4	3
`clean_quantile: 0.999` + empirisch	248 (180)	30/30	38,9	2
`clean_quantile: 0.999` + robust 0,9 + empirisch	317 bis 444	30/30	41,6 bis 45,3	3 bis 4

Die Bereinigung dauerte bei rund 10.000 Referenzpunkten 0,2 Sekunden (klassisch) bzw. etwa 4 Sekunden (robust).

Annahmen, Grenzen und Empfehlungen

Das Modell ist so gut wie seine Referenz: Es erkennt Abweichungen vom Referenzbetrieb, nicht Fehler an sich.

Annahmen und Grenzen:

- **Referenz = Normalbetrieb:** Eine länger anhaltende Störung wird von der gleitenden Referenz nach und nach als normal gelernt, nach spätestens 7 Tagen vollständig. Regel E (Drift) und eine feste Referenz wirken dem entgegen.
- **Lineare Zusammenhänge:** Die Kovarianz erfasst nur lineare Beziehungen zwischen Merkmalen. Nichtlineare Zusammenhänge werden nur teilweise abgebildet.
- **Ein Betriebszustand je Referenz:** Wechselt eine Instanz zwischen deutlich verschiedenen Zuständen (z. B. Batchbetrieb nachts, interaktiv tagsüber), mischt das Modell sie zu einer breiten Verteilung und wird in beiden Zuständen unempfindlicher.
- **Normalverteilung bei `chi2`:** Die Fehlalarmrate von 1 % gilt nur bei annähernd normalverteilten Daten. Server-Metriken mit Tagesgang und schiefen Verteilungen (Bytes/s) weichen davon ab.
- **Unabhängige Zeitpunkte:** Jeder Messzeitpunkt wird für sich bewertet; der zeitliche Verlauf fließt nur über `rate()`-Fenster und die Alert-Regeln ein.
- **Punktwerte:** Die Daten werden als Werte zu den Rasterzeitpunkten gelesen, nicht als Mittelwerte über das Raster. Kurze Spitzen zwischen zwei Rasterpunkten können fehlen.
- **Datenmenge:** Die Schätzung braucht deutlich mehr Referenzpunkte als Merkmale. Mit 7 Merkmalen und rund 10.000 Punkten (7 Tage, 1 Minute) ist das erfüllt.

Keine aktuellen Messpunkte in der Referenz

Die Referenz darf keine Messpunkte unmittelbar vor dem bewerteten Zeitpunkt enthalten (t−1, t−2, t−3 usw.). Sonst lernt das Modell eine anhaltende Störung als normal und maskiert sie selbst. Deshalb endet die Referenz mit `end_offset` 1 Stunde vor jetzt.

Der Grund: In Richtung der Anomalie hat der Normalbetrieb fast keine Streuung. Wenige gestörte Punkte in der Referenz blähen genau diese Varianz auf. Näherungsweise gilt für k solche Punkte in einer Referenz mit n Punkten:

```
D^2_{\mathrm{neu}} \approx \frac{D^2}{1 + k\,D^2/n} \qquad D_{\mathrm{neu}} < \sqrt{n/k}
```

Unabhängig von der Stärke der Störung kann D die Grenze √(n/k) nicht überschreiten. Test mit `q556d` (30 Minuten Störung, Schwelle 4,30), Referenz jeweils ergänzt um t−1 bis t−3:

Referenz	Obergrenze D	D Median ohne / mit t−1…t−3	erkannt mit t−1…t−3
7 Tage (n ≈ 10.000)	≈ 58	38,0 / 32,1	30 von 30
2 Stunden (n ≈ 120)	≈ 6,4	39,0 / 7,8	30 von 30
30 Minuten (n ≈ 30)	≈ 3,2	49,7 / 6,0	26 von 30

Mit langer Referenz bleibt die Erkennung erhalten, D wird aber verzerrt, und zwar umso stärker, je größer die Störung ist. Mit kurzer Referenz liegt die Obergrenze nahe oder unter der Schwelle, und anhaltende Störungen werden nicht mehr zuverlässig erkannt. Bei `rate()`-Merkmalen ist der Effekt besonders stark, weil t−1 bis t−3 wegen des 5-Minuten-Fensters fast identisch mit dem bewerteten Punkt sind.

Für Versuche ist die Ergänzung als Option `reference.include_recent: N` umgesetzt (Standard 0, siehe Betrieb des Service). Im Test mit N = 3 und 7 Tagen Referenz sank D während der Störung auf q556d im Median von 38,0 auf 32,1; erkannt wurden weiterhin alle Störungsminuten.

Alternativen je nach Ziel:

- **Anpassung an den aktuellen Betriebszustand:** gleitende 7-Tage-Referenz für langsame Veränderungen, getrennte Referenz je Betriebszustand für deutlich verschiedene Zustände.
- **Erkennung plötzlicher Sprünge:** die Veränderung als zusätzliches Merkmal aufnehmen (als Spalte), z. B. `deriv(system_load5[5m])`. Verzögerte Rohwerte als Spalten wären wegen der Autokorrelation fast kollinear und machen die Kovarianz instabil.

Feste oder gleitende Referenz

Eine feste Referenz in der Vergangenheit löst das Problem der Selbstmaskierung nicht anders als die gleitende: Wie stark zusätzliche aktuelle Punkte maskieren, hängt nur von der Größe n der Referenz ab, nicht davon, wann sie liegt. Aktuelle Punkte gleichen auch keine Veränderung seit der Referenz aus. Die Dämpfung $1/(1 + k \cdot D^2/n)$ trifft große Abweichungen stark und kleine wie eine Drift kaum.

Test mit dauerhafter Pegelverschiebung (Speicher seit 2 Tagen +6 Prozentpunkte), bewertet über 6 Stunden:

Referenz	Fehlalarme q556 ohne / mit t−1…t−3	Störung q556d erkannt
gleitend 7 Tage, bis 1 h vor jetzt	2 / 2	30 von 30
fest 7 Tage, endet vor 1 Tag	6 / 6	30 von 30
fest 1 Tag, vor 5 Tagen	310 / 301	30 von 30

Ohne Veränderung seit der Referenz verhalten sich alle drei Varianten gleich: 1 Fehlalarm in 6 Stunden, Störung vollständig erkannt.

Referenz	Vorteil	Nachteil
gleitend, `end_offset: 1h` (Standard)	passt sich gewollten Änderungen innerhalb weniger Tage an	lernt Störungen, die länger als einige Stunden anhalten, nach und nach als normal
gleitend, `end_offset: 1d`	Störungen unter 1 Tag gelangen nie in die Referenz; dauerhafte Änderungen werden nach 1 Tag schrittweise übernommen	nach einer gewollten Änderung 1 bis 2 Tage erhöhte Fehlalarme
fest (`reference.start`/`end`)	Basis ändert sich nie, Ergebnisse über Wochen vergleichbar	nach jeder gewollten Änderung Dauer-Fehlalarme, bis die Referenz neu gesetzt wird

Jede Referenz sollte mindestens 7 Tage umfassen, damit Tag und Nacht sowie Werktag und Wochenende enthalten sind. Ein einzelner Tag liefert nur rund 1.440 Punkte und eine zu schmale Verteilung.

Empfehlung: gleitend 7 Tage mit `end_offset: 1d` als Mittelweg. Eine feste Referenz nur dort, wo Änderungen kontrolliert durchgeführt werden und die Referenz danach bewusst neu gesetzt wird.

Empfehlungen:

1. Zähler immer mit `rate()` in die Merkmals-Queries aufnehmen.
2. Bei Servern mit Tagesgang `schwelle: empirisch` verwenden.
3. Die gleitende 7-Tage-Referenz mit end_offset 1 Tag verwenden. Eine feste Referenz nur bei kontrolliert durchgeführten Änderungen, dann nach jeder geplanten Änderung (Hardware, Software, Last) neu setzen.
4. Kann die Referenz Störungen enthalten, die Bereinigung mit clean_quantile 0.999 einschalten. Die robuste Schätzung nur ergänzend einsetzen, dann mit `support_fraction: 0.9` und `schwelle: empirisch`.
5. Die Alert-Schwellen nach einigen Tagen Betrieb anhand der Fehlalarme im Dashboard nachjustieren.

Konfigurationsreferenz

Alle Parameter der `config.yml` mit ihren Standardwerten. Dauern werden als `30s`, `5m`, `1h`, `7d` angegeben; `${VARIABLE}` wird durch Umgebungsvariablen ersetzt.

Parameter	Standard	Bedeutung
`victoriametrics.read_url`	`http://localhost:8428`	Basis-URL zum Lesen (Cluster: vmselect mit `/select/0/prometheus`)
`victoriametrics.write_url`	`…:8428/api/v1/import/prometheus`	Import-URL zum Schreiben (Cluster: vminsert)
`victoriametrics.username` / `password` / `bearer_token`	leer	Authentifizierung
`victoriametrics.timeout`	`60s`	Zeitlimit je Anfrage
`victoriametrics.verify_tls`	`true`	TLS-Prüfung: true, false oder Pfad zu CA-Bundle
`victoriametrics.max_points_per_query`	`10000`	Punkte je Zeitreihe und Teilabfrage
`schedule.interval`	`5m`	Abstand der Läufe
`schedule.step`	`1m`	Auflösung der Daten und Ergebnisse
`schedule.delay`	`1m`	Abstand des Messfenster-Endes zu jetzt
`schedule.max_catchup`	`1h`	maximales Nachholen nach Unterbrechung
`schedule.state_file`	leer	Datei für den

Parameter	Standard	Bedeutung
		Verarbeitungsstand je Instanz
`reference.window`	`7d`	Länge der gleitenden Referenz
`reference.end_offset`	`1h`	Abstand des Referenz-Endes zu jetzt
`reference.start` / `end`	leer	feste Referenz, ersetzt `window` und `end_offset`
`reference.step`	wie `schedule.step`	Auflösung der Referenz
`reference.refresh`	`1h`	Neuladen der gleitenden Referenz
`reference.min_points`	`100`	Mindestzahl vollständiger Referenzpunkte je Instanz
`reference.include_recent`	`0`	Referenz je Zeitpunkt t um t−1 bis t−N ergänzen; 0 = aus
`reference.clean_quantile`	leer (aus)	Referenz iterativ bereinigen: Punkte mit D über diesem χ^2-Quantil entfernen, empfohlen 0.999
`reference.clean_iterations`	`3`	maximale Zahl der Bereinigungsdurchläufe
`analysis.quantil`	`0.99`	Quantil für die Schwelle
`analysis.schwelle`	`chi2`	`chi2` oder `empirisch`
`analysis.robust`	`false`	robuste Schätzung (MCD)
`analysis.support_fraction`	leer (≈ 0,5)	Anteil der Referenz im MCD-Kern, empfohlen 0.9
`group_labels`	`[instance]`	Labels, nach denen getrennt bewertet wird
`metrics`	–	Merkmalsname → PromQL-Query, eine Zeitreihe je Instanz
`output.labels`	leer	feste Labels für alle Ergebnis-Metriken (ohne `instance`)
`output.prefix`	`mahalanobis`	Präfix der Metriknamen
`output.mit_messwert`	`false`	zusätzlich

Parameter	Standard	Bedeutung
		`mahalanobis_messwert` schreiben
`output.write_input`	`false`	Eingangswerte zusätzlich schreiben
`output.input_prefix`	`mahalanobis_input_`	Präfix der Eingangswerte
`output.service_metrics`	`true`	Service-Metriken schreiben